Pipeline de propensión de compra de bicicletas

sales_data.csv → data_processing → model_training → model_evaluation → data/08_reporting (Kedro)

1. sales_data.csv (data/01_raw)

- Datos crudos de ventas con 18 columnas. Se leen como **pandas.CSVDataset** y nunca se modifican.
- Cada etapa guarda su salida en una capa de Kedro (02_intermediate … 08_reporting).

2. data_processing

- **clean_sales**: valida las 18 columnas, parsea fechas, elimina faltantes y duplicados exactos, y corrige categoría/subcategoría con la **moda por producto**.
- Inconsistencias de costo/ganancia, ganancias negativas y outliers de edad (IQR) solo se **reportan** en cleaning_report.json.
- **keep_stable_catalog_months**: conserva meses con todas las categorías y cuyo mes anterior también las tenía (excluye el periodo solo-bicicletas y jul-2013).
- **build_model_input**: predictores *edad, genero, pais, estado, mes*; target *es_bicicleta* (Product_Category == Bikes). Excluye producto, precio, costo, cantidad e ingreso para evitar fuga de información.
- Crea **grupo** (ngroup de día, mes, edad, género, estado) para identificar registros “gemelos”.

3. model_training

- **split_data**: GroupShuffleSplit 80/20 (random_state=42) por *grupo*; ningún gemelo queda en train y test.
- **fit_baselines**: DummyClassifier(prior) y regresión logística (one-hot con min_frequency=5, SplineTransformer(6 nodos, grado 3) en edad, C=0.2). CV con StratifiedGroupKFold de 5 folds.
- **tune_lightgbm**: Optuna con sampler TPE, 60 trials, folds fijos (cv_seed=11). Maximiza la **PR-AUC media** en CV sobre 10 hiperparámetros definidos en parameters.yml.
- **LightGBM**: categóricas con OrdinalEncoder y declaradas con *categorical_feature* (sin one-hot); deterministic=True para reproducibilidad.
- **fit_final_model**: reentrena con best_params sobre todo el train.

4. model_evaluation (solo conjunto de prueba)

Nodo	Cómo se calcula
evaluate_models	ROC-AUC, PR-AUC, log-loss, Brier. IC 95 % por **bootstrap de grupos completos** (500 iteraciones, percentiles 2.5 y 97.5) para logística y LightGBM.
compute_lift_table	Ordena por score, deciles con pd.qcut y lift = tasa del decil / tasa global.
compute_business_impact	Top 10–50 % de la base: precisión, lift, % de bicicletas y de ganancia capturadas.
compute_permutation_importance	sklearn permutation_importance, scoring ROC-AUC, 10 repeticiones.
compute_segment_summary	País × tramo de edad × género; tasa de bicicleta y ganancia media; mínimo 300 órdenes.
make_diagnostic_plots	ROC, calibración (10 bins), lift por decil, curva de ganancias e importancia.

5. data/08_reporting

- Salida final: **metrics.json**, lift_table.csv, business_impact.csv, permutation_importance.csv, segment_summary.csv y plots/ (PNG, 150 dpi).
- best_params.json y tuning_trials.csv quedan en 06_models; test_predictions en 07_model_output.